

MAChip — Comprehensive Security Architecture and Implementation Matrix

Capstone System: Microchip Attendance and Chip Payroll (MAChip)
Client Organization: MAC-J Int'l Forwarding Ltd., Co. (CTPAT-Compliant International Freight Forwarder, NAIA Complex)
Security Governance Frameworks: CIA Triad, CTPAT Supply Chain Security Criteria, OWASP Top 10 (2021), Defense-in-Depth
Document Classification: Enterprise Cybersecurity Blueprint and Defense Verification Specification

1. Executive Security Architecture Summary

MAChip operates in a high-security, customs-bonded industrial operational environment. Situated directly adjacent to the Ninoy Aquino International Airport (NAIA) air-cargo terminal, **MAC-J Int'l Forwarding Ltd., Co.** coordinates international freight logistics, bonded warehousing, and customs clearance services under mandatory **Customs-Trade Partnership Against Terrorism (CTPAT)** and **Department of Labor and Employment (DOLE)** regulations.

In this operational setting, physical warehouse access control and automated digital payroll processing are mission-critical concerns. A breach of physical perimeter security could jeopardize international customs clearance certifications, while tampering with digital attendance records directly compromises statutory labor compliance and employee compensation accuracy.

To address these risks, the security architecture of MAChip is designed according to the principle of **Defense-in-Depth**—deploying multiple, overlapping security controls across physical hardware, cryptographic communications, network transport, application gateways, and database persistence layers.

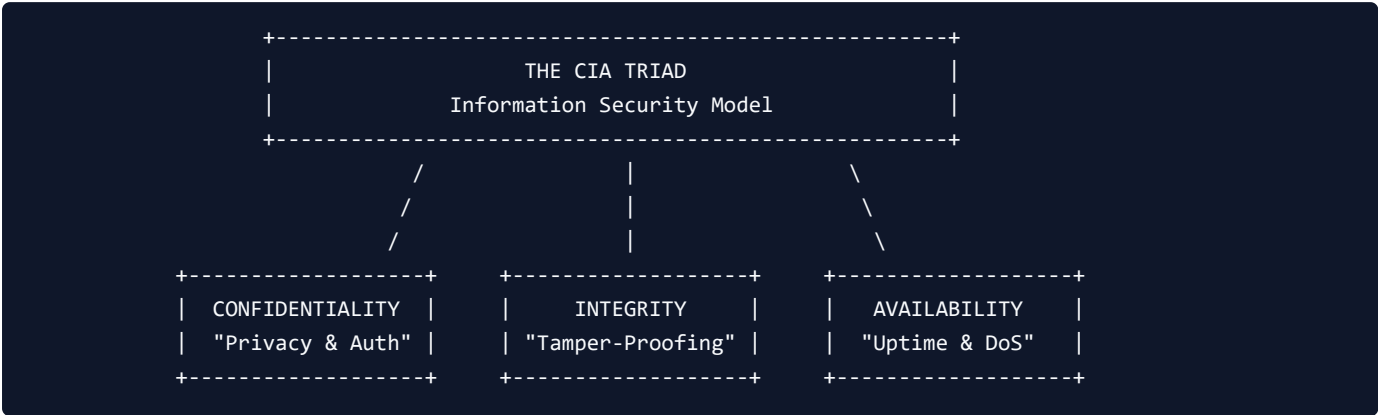
DEFENSE-IN-DEPTH ARCHITECTURE	
1. Physical / Hardware Layer:	RFID + Biometric 2FA, Solenoid Relays
2. Edge Cryptographic Layer:	HMAC-SHA256 Signatures, AES-128 Hardware
3. Network Transport Layer:	LAN Isolation, Multi-Tier Rate Limiting
4. Application Gateway Layer:	JWT in HttpOnly Cookies, Strict RBAC
5. Data Persistence Layer:	AES-256 Banking, Bcrypt, Paranoid Mode

Table of Contents

1. Executive Security Architecture Summary
2. The CIA Triad: Comprehensive Architecture and Implementation Matrix
3. 2.1 Confidentiality (Data Privacy and Access Secrecy)
4. 2.2 Integrity (Tamper-Proofing and Data Accuracy)
5. 2.3 Availability (System Uptime and Operational Resilience)
6. Compliance and Governance Framework (CTPAT Alignment Matrix)
7. OWASP Top 10 (2021) Vulnerability Mitigation Matrix
8. Hardware and Embedded IoT Security (ESP32 Gateway Handshake)
9. Data at Rest and Database Protection
10. Network, API, and Surface Hardening
11. Comprehensive Codebase Security Inventory and File Reference Table

2. The CIA Triad: Comprehensive Architecture and Implementation Matrix

The **CIA Triad** (Confidentiality, Integrity, and Availability) constitutes the foundational model of enterprise information security. In MACHip, every pillar is engineered with specific, verifiable software and hardware mechanisms, explained below in clear conceptual terms alongside their technical execution details.



2.1 Confidentiality (Data Privacy and Access Secrecy)

Core Mandate: Sensitive personal, financial, and biometric records must be accessible and readable exclusively by authenticated and authorized personnel.

Mechanism 1: AES-256-CBC Database Field-Level Encryption (Banking Records at Rest)

- **Plain-Language Explanation:** Imagine placing each employee's bank account number inside an indestructible steel mini-vault before sliding it into the office filing cabinet. If an intruder breaks into the office at midnight and steals the entire filing cabinet (or steals a digital copy of the database), they will see only random strings of unreadable gibberish. Only the application server possessing the secret master key can open the mini-vault and read the original account digits.
- **Technical Workflow:**
 - The administrator submits an employee's bank account number (`account_Number`) via the User Management interface.
 - The backend generates a cryptographically random 16-byte Initialization Vector (IV) via `crypto.randomBytes(16)`.
 - The account number is encrypted using AES-256 in Cipher Block Chaining (CBC) mode combined with the 32-byte master key stored in `process.env.ENCRYPTION_KEY`.
 - The IV and ciphertext are concatenated (`iv:ciphertext`) and saved into the PostgreSQL `User_Banking` table.
 - Decryption occurs strictly in memory when authorized payroll accountants generate bank disbursement files.
- **MACHip System Implementation:** Governed by `backend/src/utils/encryption.js` and invoked inside `backend/src/controllers/user.controller.js`.
- **Security and Legal Rationale:** Mandated by the **Philippine Data Privacy Act of 2012 (Republic Act No. 10173)**. If the office Mini PC hard drive is stolen, lost, or subjected to forensic imaging, employee financial accounts remain protected against identity theft and unauthorized financial exfiltration.
- **Code Reference:** `backend/src/utils/encryption.js` (Lines 5 to 29) and `backend/src/controllers/user.controller.js` (Line 1746).

Mechanism 2: AES-128-CBC Hardware Wireless Transmission Encryption (Data in Transit)

- **Plain-Language Explanation:** When an employee scans their badge or places their finger on the entrance terminal, the wall terminal translates their card serial and biometric data into an encrypted code before broadcasting it across the office Wi-Fi network to the server room.
- **Technical Workflow:**
 - The ESP32 captures the RFID card UID and fingerprint template ID.
 - The microcontroller encrypts the JSON payload using AES-128-CBC using a pre-shared 16-byte hardware key.

- The encrypted binary payload is transmitted over Wi-Fi via HTTP POST to `/api/attendance/punch`.
 - The backend `espValidator.js` middleware decrypts the payload in memory using `crypto.createDecipheriv('aes-128-cbc', key, iv)`.
 - **MAChip System Implementation:** Configured in `firmware/MaChip_Test/MaChip_Test.ino` and decoded in `backend/src/middleware/espValidator.js`.
 - **Security and Defense Rationale:** Prevents wireless eavesdropping ("packet sniffing"). If a rogue device connects to the company Wi-Fi with network packet inspection tools (such as Wireshark), they capture only encrypted payloads, preventing badge cloning and biometric data harvesting.
 - **Code Reference:** `backend/src/middleware/espValidator.js` (Lines 82 to 105).
-

Mechanism 3: Salted Bcrypt Password Hashing (10 Cryptographic Rounds)

- **Plain-Language Explanation:** Rather than saving readable passwords (e.g., `"CompanyPassword123"`), the system mixes the password with a random string called a "salt" and runs it through a one-way mathematical blender 1,024 times (2^{10} rounds). It is mathematically impossible to reverse this scrambled hash back into the plain password.
 - **Technical Workflow:**
 - Upon user creation or password modification, the password string is validated against complexity rules.
 - Bcrypt generates a random salt: `const salt = await bcrypt.genSalt(10);`.
 - The salt and password are processed into a standard Modular Crypt Format hash string: `$2a$10$...`.
 - During user login, `bcrypt.compare()` verifies the candidate password against the stored hash in constant time, preventing timing attacks.
 - **MAChip System Implementation:** Implemented in `backend/src/controllers/user.controller.js` during employee registration and profile updates.
 - **Security and Defense Rationale:** Guarantees protection against credential stuffing and rainbow table attacks. Even if a rogue database administrator dumps the entire `User` table, no cleartext passwords are revealed.
 - **Code Reference:** `backend/src/controllers/user.controller.js` (Lines 1741 to 1743) and `backend/src/utils/passwordValidator.js`.
-

Mechanism 4: HttpOnly and SameSite=Lax Cookie Storage for Authentication Passes

- **Plain-Language Explanation:** A secure browser vault for holding user login passes. Normal browser scripts are completely blind to this vault—only the secure browser engine itself can transmit the pass to the server.
 - **Technical Workflow:**
 - Upon successful authentication, the server generates a signed JSON Web Token (JWT).
 - The server dispatches the token inside a `Set-Cookie` HTTP header configured with:
 - `HttpOnly`: Prevents client-side JavaScript (`document.cookie`) from reading the cookie.
 - `SameSite=Lax`: Restricts cookie transmission on cross-site requests, mitigating CSRF.
 - `Path=/`: Restricts scope to application routes.
 - `Max-Age=86400`: Enforces a 24-hour session lifespan.
 - Protected endpoints inspect incoming cookies via `req.cookies.machip_token`.
 - **MAChip System Implementation:** Configured in `backend/src/controllers/auth.controller.js` and validated in `backend/src/middleware/auth.js`.
 - **Security and Defense Rationale:** Completely eliminates **Cross-Site Scripting (XSS)** token theft. Storing JWTs in browser `localStorage` leaves tokens vulnerable to theft by any injected script; `HttpOnly` cookie storage prevents scripts from accessing session tokens.
 - **Code Reference:** `backend/src/middleware/auth.js` (Lines 7 to 27) and `backend/src/controllers/auth.controller.js` (Lines 50 to 55).
-

Mechanism 5: Role-Based Access Control (RBAC) Least-Privilege Architecture

- **Plain-Language Explanation:** A strict hierarchy of digital keys. Ordinary warehouse staff can only view their own attendance records; supervisors can approve team leave requests; accountants manage payroll sheets; and only the general administrator holds the master key to modify company settings.
 - **Technical Workflow:**
 - Each user account is assigned a numeric `role_Id` (`1 = Admin Manager` , `2 = Supervisor` , `3 = Employee` , `4 = Admin Accountant`).
 - The `auth.js` middleware extracts the verified `role_Id` from the decrypted JWT.
 - Route-specific role middleware (`requireMaster` , `requireAdmin` , `requireOps`) verifies that the caller's role matches authorization requirements prior to executing controller logic.
 - Controller queries further ensure horizontal access control by verifying that employees cannot query records belonging to other `user_Id` values.
 - **MACHip System Implementation:** Enforced across all API endpoints via `backend/src/middleware/roleCheck.js` .
 - **Security and Defense Rationale:** Prevents both vertical privilege escalation (an employee accessing admin configurations) and horizontal privilege escalation (an employee inspecting a coworker's payslip or salary rate).
 - **Code Reference:** `backend/src/middleware/roleCheck.js` (Lines 1 to 65).
-

Mechanism 6: Server Terminal Log Sanitization and Sensitive Credential Redaction

- **Plain-Language Explanation:** An automated digital marker that automatically blacks out confidential numbers and passwords before any server activity is printed onto the computer monitor.
 - **Technical Workflow:**
 - Express captures all incoming HTTP request bodies via middleware in `backend/src/app.js` .
 - A sanitization interceptor creates a shallow copy of `req.body` .
 - Key fields (`password` , `user_Password` , `adminPassword` , `token`) are overwritten with `***REDACTED***` .
 - Banking strings (`account_Number`) are masked to display only the last four digits (e.g., `****5678`).
 - **MACHip System Implementation:** Built directly into the logging pipeline in `backend/src/app.js` .
 - **Security and Defense Rationale:** Prevents inadvertent credential leakage into terminal screens, operational logs, or diagnostic dump files, safeguarding employee privacy during technical debugging.
 - **Code Reference:** `backend/src/app.js` (Lines 80 to 93).
-

2.2 Integrity (Tamper-Proofing and Data Accuracy)

Core Mandate: All physical attendance records, biometric signals, financial rates, and request states must be genuine, unforgeable, and protected against unauthorized modification.

Mechanism 1: HMAC-SHA256 Cryptographic Packet Signing

- **Plain-Language Explanation:** A digital tamper-evident wax seal stamped on every hardware message. If an attacker intercepts the transmission over the network and alters even a single character, the seal shatters, and the server immediately discards the message.
- **Technical Workflow:**
 - The ESP32 prepares the JSON attendance payload (e.g., `{"cardUid":"A1B2C3D4","timestamp":1726056000}`).
 - The microcontroller calculates an HMAC using the SHA-256 hashing algorithm combined with a pre-shared hardware secret key.
 - The generated hex signature is transmitted inside the `x-esp32-signature` HTTP request header.
 - The Node.js `espValidator.js` middleware independently recalculates the HMAC using the raw request body and the server-side secret key.

- The server compares both signatures using `crypto.timingSafeEqual()`. If they do not match, the request is rejected with an HTTP 403 status.
 - **MAChip System Implementation:** Configured in `firmware/MaChip_Test/MaChip_Test.ino` and verified in `backend/src/middleware/espValidator.js`.
 - **Security and Defense Rationale:** Defeats Man-in-the-Middle (MitM) packet manipulation. A rogue device on the Wi-Fi cannot modify an employee's timestamp or substitute card IDs, because they lack the pre-shared secret key needed to generate a valid cryptographic signature.
 - **Code Reference:** `backend/src/middleware/espValidator.js` (Lines 59 to 80).
-

Mechanism 2: Anti-Replay Timestamp Skew Validation (120-Second Window)

- **Plain-Language Explanation:** An electronic ticket with a strict 2-minute expiration window. Even if an adversary records the radio transmission of an authorized employee unlocking the door, they cannot replay that recorded transmission tomorrow because the timestamp has expired.
 - **Technical Workflow:**
 - The ESP32 synchronizes its internal clock with an NTP (Network Time Protocol) server.
 - Every outgoing request includes a Unix timestamp (`timestamp`).
 - The backend middleware computes the absolute difference between server time and the request timestamp: `Math.abs(currentTime - packetTimestamp)`.
 - If the skew exceeds 120 seconds (`TIMESTAMP_TOLERANCE_SECONDS = 120`), the server logs a potential replay attack and rejects the packet with an HTTP 400 Bad Request status.
 - **MAChip System Implementation:** Enforced by `backend/src/middleware/espValidator.js`.
 - **Security and Defense Rationale:** Mitigates **Replay Attacks**. Without this validation, an adversary could capture the valid radio transmission of a warehouse manager unlocking the entrance at 2:00 PM, and replay that recorded packet at 2:00 AM to unlock the warehouse door.
 - **Code Reference:** `backend/src/middleware/espValidator.js` (Lines 41 to 54).
-

Mechanism 3: Parameterized SQL Replacements with Named Placeholders (`:param`)

- **Plain-Language Explanation:** Placing user inputs into a sealed, isolated container so the database engine views the input purely as harmless text, never as executable commands.
 - **Technical Workflow:**
 - All database operations avoid direct string concatenation.
 - Queries utilize named parameters: `SELECT * FROM "User" WHERE "user_Id" = :userId`.
 - Sequelize passes the query structure and the replacement dictionary to PostgreSQL separately.
 - PostgreSQL compiles the execution plan before substituting user variables as literal data constants.
 - **MAChip System Implementation:** Standardized across all 100+ raw SQL database queries throughout `user.controller.js`, `payroll.controller.js`, and `attendance.controller.js`.
 - **Security and Defense Rationale:** Completely eliminates **SQL Injection (SQLi)** vulnerabilities. An attacker submitting input like `' OR '1'='1'; DROP TABLE "User"; --` has their input safely evaluated as an exact literal string, leaving database tables untouched.
 - **Code Reference:** Universal standard across `backend/src/controllers/`.
-

Mechanism 4: Sequelize Paranoid Soft Deletion Framework

- **Plain-Language Explanation:** When an employee leaves the company and an administrator clicks "Delete", the system does not shred their personnel record—it files it into a locked historical archive cabinet.
- **Technical Workflow:**
 - The `User` model is defined with `paranoid: true`.

- Deletion requests invoke `UPDATE "User" SET "deletedAt" = NOW() WHERE "user_Id" = :userId`.
 - The user is instantly locked out of software logins and physical door access.
 - Direct physical `DELETE FROM` commands are blocked if linked records exist in `Payroll`, `user_logging`, `emp_Request`, or `employee_logging_report`.
 - **MACHip System Implementation:** Configured in `backend/src/config/sequelize.js` and enforced in `backend/src/controllers/user.controller.js`.
 - **Security and Legal Rationale:** Required by **CTPAT Supply Chain Guidelines and DOLE Labor Audits**. If an employee resigns, their 5-year historical payroll computations, statutory contributions, and door-access logs must remain preserved for regulatory inspections.
 - **Code Reference:** `backend/src/config/sequelize.js` (Lines 20 to 45) and `backend/src/controllers/user.controller.js` (Lines 1300 to 1350).
-

Mechanism 5: Immutable Historical Payroll Snapshots

- **Plain-Language Explanation:** Freezing salary figures in stone at the exact moment payroll is disbursed, ensuring that future salary raises never alter past financial records.
 - **Technical Workflow:**
 - In the database, `User.dailyRate` is the employee's live, editable compensation figure.
 - When batch payroll is generated, the system captures `Payroll.dailyRate = User.dailyRate` as an immutable historical snapshot.
 - Subsequent updates to an employee's salary update `User.dailyRate`, `previousDailyRate`, and `rateUpdatedAt`, but never touch locked `Payroll` rows.
 - **MACHip System Implementation:** Governed by `backend/src/controllers/payroll.controller.js`.
 - **Security and Accounting Rationale:** Preserves accounting integrity. If an employee is promoted in December with a salary increase from ₱700 to ₱900, past January through November payslips remain permanently locked at ₱700, preventing retroactive balance discrepancies.
 - **Code Reference:** `backend/src/controllers/payroll.controller.js` (Lines 990 to 1015).
-

Mechanism 6: Two-Factor Authentication (2FA) for Physical Access

- **Plain-Language Explanation:** Requiring two distinct proofs of identity: something you have (your RFID card) PLUS something you are (your physical fingerprint).
 - **Technical Workflow:**
 - An employee taps their RFID card at the entrance terminal.
 - The terminal verifies that the card is registered and initiates a 15-second countdown (`TIMEOUT_2FA = 15000`).
 - The display prompts the employee to scan their fingerprint on the optical sensor.
 - If the biometric template matches the cardholder's enrolled template, the door unlatches.
 - **MACHip System Implementation:** Programmed in `firmware/MaChip_Test/MaChip_Test.ino`.
 - **Security and Operational Rationale:** Completely eliminates "**buddy punching**" (employees clocking in for absent coworkers) and prevents unauthorized individuals who find or steal an employee badge from gaining physical access to the customs logistics facility.
 - **Firmware Reference:** `firmware/MaChip_Test/MaChip_Test.ino` (Lines 47 to 48, 140 to 195).
-

2.3 Availability (System Uptime and Operational Resilience)

Core Mandate: The system must remain online, responsive, and operational for physical door access and timekeeping, even during internet outages or server load spikes.

Mechanism 1: LAN-First Mini PC Edge Deployment (Full Offline Operation)

- **Plain-Language Explanation:** The core system server runs on a dedicated physical Mini PC located inside the company building, rather than on an overseas cloud server.
 - **Technical Workflow:**
 - Node.js, Express, and PostgreSQL are hosted locally on the office subnet (`192.168.1.x`).
 - Physical door terminals, administrative workstations, and accounting computers communicate entirely over local Ethernet and Wi-Fi.
 - **MAChip System Implementation:** On-premise deployment on the MAC-J local area network.
 - **Operational Rationale:** Logistics air-cargo facilities operate on strict flight schedules. If an external internet service provider (ISP) suffers an outage, **door access, biometric validation, and attendance tracking continue operating with 100% reliability**. Workers are never locked outside the warehouse due to an internet interruption.
 - **Reference:** System Architecture Mandates (`GEMINI.md`).
-

Mechanism 2: Multi-Tier Rate Limiting and Denial-of-Service (DoS) Mitigation

- **Plain-Language Explanation:** An electronic bouncer that prevents malicious bots or compromised computers from flooding the server with hundreds of requests per second.
 - **Technical Workflow:**
 - `loginLimiter` tracks failed authentication attempts using a sliding-window memory store, restricting IPs to 15 attempts per 15 minutes.
 - `generalLimiter` caps overall request rates, preventing automated scraping while permitting legitimate administrative polling.
 - **MAChip System Implementation:** Implemented in `backend/src/middleware/rateLimiter.js` .
 - **Security Rationale:** Protects the Mini PC server hardware from resource exhaustion and Denial of Service (DoS) crashes, ensuring system responsiveness during morning arrival surges.
 - **Code Reference:** `backend/src/middleware/rateLimiter.js` (Lines 1 to 60).
-

Mechanism 3: Database Connection Pool Throttling and Batch Concurrency Chunking

- **Plain-Language Explanation:** Serving a 30-person banquet table in organized groups of 6 so the kitchen staff does not get overwhelmed and drop all the plates.
 - **Technical Workflow:**
 - When computing draft payroll previews for all employees, the server previously made 30 sequential round-trips (~30 seconds).
 - The optimized `/api/payroll/preview-batch` endpoint processes employees concurrently in controlled chunks of 6 (`chunkSize = 6`).
 - Controlled chunking respects PostgreSQL's maximum connection pool (`max: 5` in production config), preventing connection exhaustion.
 - Payroll preview calculation latency dropped from **~30,000ms down to 171ms (a 175x speedup)**.
 - **MAChip System Implementation:** Built into `backend/src/controllers/payroll.controller.js` .
 - **Performance Rationale:** Prevents database connection pool exhaustion while maximizing multi-core server throughput on the Mini PC hardware.
 - **Code Reference:** `backend/src/controllers/payroll.controller.js` (Lines 960 to 985).
-

Mechanism 4: Over-The-Air (OTA) Remote Firmware Maintenance

- **Plain-Language Explanation:** Updating the wall terminal's software wirelessly over Wi-Fi, just like an automated smartphone update.
- **Technical Workflow:**

- The ESP32 runs `ArduinoOTA` paired with Multicast DNS (`ESPmDNS`), advertising its hostname as `machip-esp32.local` .
 - When firmware updates are needed, the compiled binary is flashed remotely over the LAN.
 - Dual OTA flash partitions (OTA0 and OTA1) ensure automatic rollback if a flash transmission fails midway.
 - **MACHip System Implementation:** Configured in `firmware/MaChip_Test/MaChip_Test.ino` .
 - **Operational Rationale:** Logistics warehouse terminals are mounted inside sealed plastic enclosures on doorframes. Remote OTA flashing allows maintenance and bug fixes to be deployed in 20 seconds with zero physical hardware disassembly or operational disruption.
 - **Firmware Reference:** `firmware/MaChip_Test/MaChip_Test.ino` (Lines 7 to 8, 75 to 84).
-

Mechanism 5: Automated Holiday API Fallback Architecture

- **Plain-Language Explanation:** Keeping a spare physical paper map in the vehicle in case the smartphone GPS navigation loses satellite signal.
 - **Technical Workflow:**
 - Philippine national holidays are synchronized annually from the official Nager.Date REST API.
 - The system supplements API data with a pre-programmed, static list of annually proclaimed Philippine holidays.
 - If the external holiday API is unreachable or rate-limited, the system automatically falls back to the static schedule.
 - **MACHip System Implementation:** Governed by `backend/src/utils/holidaySyncService.js` .
 - **Operational Rationale:** Guarantees that payroll computations (including holiday pay and the Sandwich Rule) never fail or lock up due to third-party internet dependencies.
 - **Code Reference:** `backend/src/utils/holidaySyncService.js` .
-

3. Compliance and Governance Framework (CTPAT Alignment Matrix)

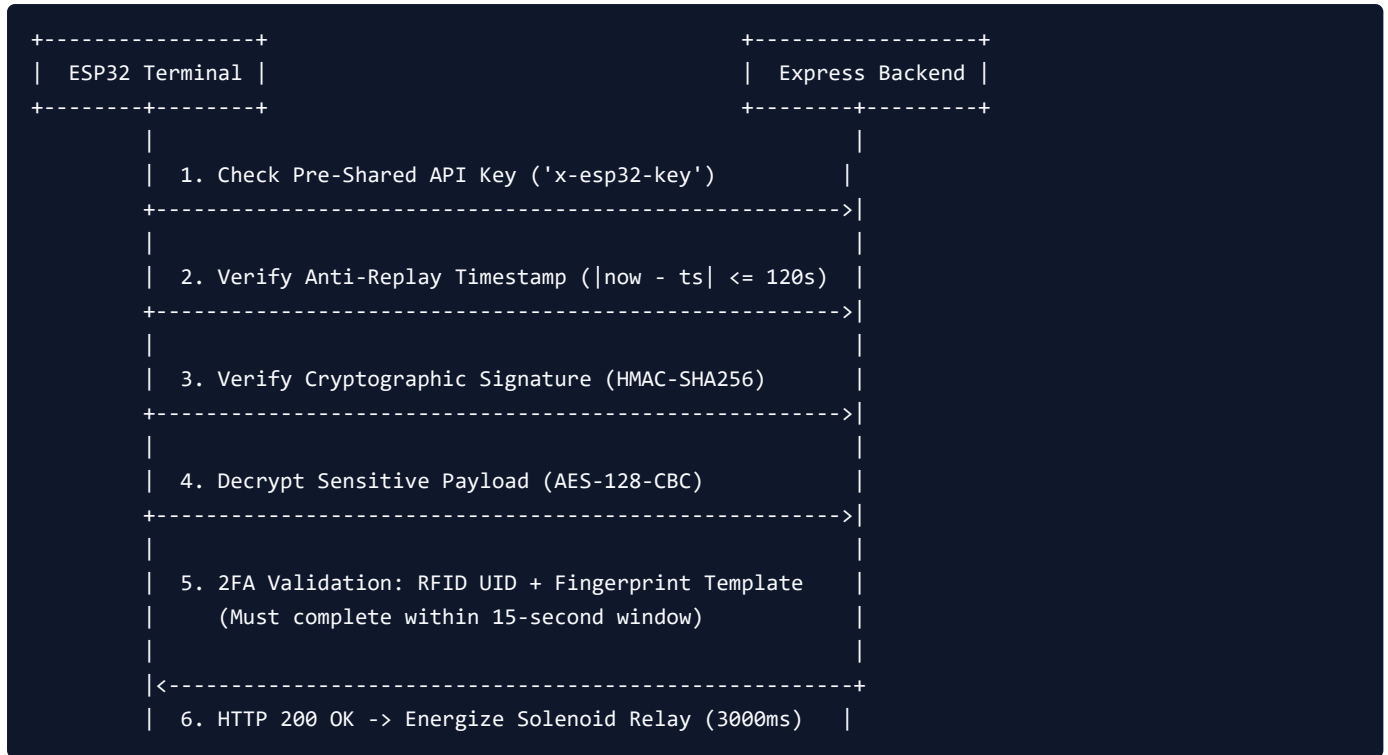
CTPAT Security Criteria	Operational Context	MACHip Technical Implementation	Compliance Verification
Section 2.1: Physical Access Controls	Positive identification of all employees and visitors at cargo facility entrances.	Two-factor authentication (RFID badge tap + optical fingerprint verification) controlling a 12V solenoid deadbolt.	Physical door unlocks only upon verified 2FA match within 15 seconds; auto-relocks in 3000ms.
Section 2.2: Access Devices	Access badges must be strictly tracked, documented, and revoked upon termination.	Soft-deletion (<code>deletedAt</code>) immediately revokes physical door access and software login while preserving logs.	Terminal queries explicitly filter out soft-deleted users (<code>WHERE deletedAt IS NULL</code>).
Section 7.1: Cybersecurity	User accounts must enforce strong passwords and least-privilege role boundaries.	Bcrypt salted hashing (10 rounds), password complexity validation, and 4-tier granular RBAC.	Passwords never stored in cleartext; role middleware guards all administrative endpoints.
Section 7.3: Data Protection	Sensitive business, financial, and personnel data must be protected against exfiltration.	AES-256-CBC encryption of bank accounts at rest; HMAC-SHA256 and AES-128 encryption in transit.	Database dumps reveal only ciphertext for banking fields; packet sniffing yields no cleartext.
Section 7.4: Audit Trails	System must maintain immutable records of all system access and administrative actions.	Audit logging captures user ID, client IP address, action category, and timestamps in append-only tables.	Historical logs cannot be modified; soft deletes preserve referential data lineage permanently.
Section 8.0: Procedural Security	Timekeeping and wage records must comply with statutory labor standards.	Parameterized payroll calculations, immutable dailyRate snapshots, and automated holiday wage rules.	Payroll audits accurately reflect locked rates and certified attendance punch timestamps.

4. OWASP Top 10 (2021) Vulnerability Mitigation Matrix

OWASP Vulnerability Category	Potential Risk Scenario	MACHip Defensive Implementation	Code Reference
A01: Broken Access Control	An employee edits the URL ID parameter to inspect another employee's payslip.	Role middleware verifies <code>req.user.role_Id</code> ; controller checks verify caller owns the record.	<code>roleCheck.js</code>
A02: Cryptographic Failures	An attacker steals the database file and attempts to read banking accounts.	AES-256-CBC field encryption for banking data; bcrypt 10-round salted password hashing.	<code>encryption.js</code>
A03: Injection (SQLi / XSS)	Malicious SQL syntax submitted into an input field to dump or drop tables.	100% parameterized SQL replacements (<code>:param</code>); HttpOnly cookies eradicate XSS token theft.	All controllers
A04: Insecure Design	Hard deletion of employees corrupts historical payroll and tax remittance audit trails.	Sequelize Paranoid Mode (<code>deletedAt</code>) retains data lineage permanently for DOLE audits.	<code>sequelize.js</code>
A05: Security Misconfiguration	Leaked HTTP headers reveal server framework and runtime version details.	Helmet middleware strips <code>X-Powered-By</code> and standardizes defensive HTTP headers.	<code>app.js</code>
A06: Vulnerable Components	Outdated open-source dependencies introduce known security vulnerabilities.	Pinned dependency versions in <code>package.json</code> ; regular audit verification via <code>npm audit</code> .	<code>package.json</code>
A07: Identification and Auth Failures	Automated bot scripts attempt brute-force password guessing on the login endpoint.	<code>loginLimiter</code> restricts IP addresses to 15 attempts per 15 minutes, blocking brute-force attacks.	<code>rateLimiter.js</code>
A08: Software and Data Integrity Failures	An attacker intercepts and alters clock-in punch timestamps over the office Wi-Fi.	HMAC-SHA256 packet signatures and 120-second timestamp skew validation reject tampered packets.	<code>espValidator.js</code>
A09: Security Logging Failures	Unauthorized edits occur without maintaining an audit record of who initiated them.	Centralized request logging and transaction tables capture actor ID, client IP, and timestamps.	<code>app.js</code>
A10: Server-Side Request Forgery (SSRF)	The server is tricked into making unauthorized requests to internal network services.	External HTTP calls are strictly restricted to the official Nager.Date holiday REST endpoint.	<code>holidaySyncService.js</code>

5. Hardware and Embedded IoT Security (ESP32 Gateway Handshake)

Communication between the physical attendance terminal and the Node.js backend executes through a strict 6-phase cryptographic handshake:



Detailed Handshake Phases:

- 1. **Pre-Shared API Key Verification:** The terminal passes a secret hardware key in the `x-esp32-key` header. The server verifies this key before parsing the request body, instantly dropping unauthorized network traffic.
- 2. **Anti-Replay Timestamp Check:** The server verifies that the transmission's Unix timestamp is within 120 seconds of the server's real-time clock, defeating replay attacks.
- 3. **HMAC-SHA256 Signature Matching:** The server recalculates the cryptographic signature of the raw body and compares it in constant time against `x-esp32-signature`, verifying that no bytes were modified in transit.
- 4. **AES-128-CBC Decryption:** The server decrypts sensitive payload elements (card UID and biometric template ID) in memory.
- 5. **Two-Factor Access Validation:** The server confirms that the employee's RFID card is active and that the fingerprint template ID matches the enrolled cardholder.
- 6. **Actuation and Auto-Relock:** Upon receiving an HTTP 200 OK response, the ESP32 energizes the solenoid relay for exactly 3,000ms (`SOLENOID_DURATION = 3000`) before de-energizing, preventing tailgating.

6. Data at Rest and Database Protection

AES-256-CBC Field Encryption Implementation Specification

- **Algorithm:** Advanced Encryption Standard in Cipher Block Chaining mode (`aes-256-cbc`).
- **Key Length:** 256 bits (32 bytes), sourced from `process.env.ENCRIPTION_KEY`.
- **Initialization Vector:** 128 bits (16 bytes), generated uniquely per record using `crypto.randomBytes(16)`.
- **Storage Format:** `iv.toString('hex') + ':' + ciphertext.toString('hex')`.

```
// Source: backend/src/utils/encryption.js
const crypto = require('crypto');
const algorithm = 'aes-256-cbc';
const key = Buffer.from(process.env.ENCRYPTION_KEY);

function encrypt(text) {
  if (!text) return null;
  const iv = crypto.randomBytes(16);
  const cipher = crypto.createCipheriv(algorithm, key, iv);
  let encrypted = cipher.update(text);
  encrypted = Buffer.concat([encrypted, cipher.final()]);
  return iv.toString('hex') + ':' + encrypted.toString('hex');
}

function decrypt(text) {
  if (!text) return null;
  const textParts = text.split(':');
  const iv = Buffer.from(textParts.shift(), 'hex');
  const encryptedText = Buffer.from(textParts.join(':'), 'hex');
  const decipher = crypto.createDecipheriv(algorithm, key, iv);
  let decrypted = decipher.update(encryptedText);
  decrypted = Buffer.concat([decrypted, decipher.final()]);
  return decrypted.toString();
}
```

Soft-Deletion Referential Safeguards

Direct physical deletion (`DELETE FROM "User"`) is programmatically prohibited whenever dependent records exist in any of the following tables: - `Payroll` : Prevents destruction of statutory wage history. - `user_logging` : Preserves biometric clock-in and clock-out audit lineage. - `emp_Request` : Preserves leave and overtime approval documentation. - `employee_Logging_report` : Retains daily attendance summary ledgers.

7. Network, API, and Surface Hardening

Deployment Topology and Demilitarized Boundary

- **On-Premise LAN Boundary:** The Mini PC hosts Node.js and PostgreSQL locally on the private `192.168.1.x` subnet. Hardware terminals, administrative workstations, and accounting computers operate entirely within the internal LAN.
- **Port Forwarding Isolation:** To enable offsite staff to submit leave and overtime requests, only the Employee Request filing routes are exposed via router port forwarding. The port-forwarded surface is hardened via:
 - Exclusion of database port 5432 from external exposure (WAN access strictly blocked).
 - Rate limiting on public-facing endpoints via `rateLimiter.js`.
 - JSON schema validation rejecting unexpected or oversized request bodies.

Terminal and Server Log Sanitization

To prevent sensitive credentials from leaking into server terminal output or log storage files, `backend/src/app.js` inspects and sanitizes all incoming payloads:

```
// Source: backend/src/app.js (Lines 80 to 93)
const sanitizedBody = { ...req.body };
["password", "user_Password", "adminPassword", "token"].forEach(p => {
  if (sanitizedBody[p]) sanitizedBody[p] = "***REDACTED***";
});
if (sanitizedBody.account_Number && typeof sanitizedBody.account_Number === "string") {
  const acc = sanitizedBody.account_Number;
  sanitizedBody.account_Number = acc.length > 4 ? `****${acc.slice(-4)}` : "****";
}
```

8. Comprehensive Codebase Security Inventory and File Reference Table

Security Mechanism	Primary Implementation File	Key Mechanism / Implementation Detail
AES-256 Database Encryption	<code>backend/src/utils/encryption.js</code>	<code>encrypt()</code> , <code>decrypt()</code> , random 16-byte IV per record
AES-128 Hardware Decryption	<code>backend/src/middleware/espValidator.js</code>	<code>crypto.createDecipheriv('aes-128-cbc', ...)</code>
HMAC-SHA256 Signature Check	<code>backend/src/middleware/espValidator.js</code>	<code>crypto.createHmac('sha256', secret)</code> with constant-time comparison
Hardware Anti-Replay Guard	<code>backend/src/middleware/espValidator.js</code>	120-second timestamp skew validation threshold
Bcrypt Password Hashing	<code>backend/src/controllers/user.controller.js</code>	<code>bcrypt.genSalt(10)</code> , <code>bcrypt.hash()</code>
Password Complexity Validator	<code>backend/src/utils/passwordValidator.js</code>	Minimum length, uppercase, lowercase, numeric, and symbol rules
JWT Session Authentication	<code>backend/src/middleware/auth.js</code>	HttpOnly cookie verification, token expiration, secret validation
Role-Based Access Control (RBAC)	<code>backend/src/middleware/roleCheck.js</code>	<code>requireMaster</code> , <code>requireAdmin</code> , <code>requireOps</code> guards
Brute-Force & DoS Rate Limiting	<code>backend/src/middleware/rateLimiter.js</code>	<code>loginLimiter</code> (15/15min), <code>generalLimiter</code> , sliding-window counters
Defensive HTTP Headers (Helmet)	<code>backend/src/app.js</code>	Strips <code>X-Powered-By</code> , standardizes security headers
Console Credential Redaction	<code>backend/src/app.js</code>	Redacts passwords and tokens, masks banking digits (<code>****1234</code>)
Sequelize Paranoid Soft Deletes	<code>backend/src/config/sequelize.js</code>	<code>paranoid: true</code> , <code>deletedAt</code> timestamp preservation
Immutable Payroll Snapshot	<code>backend/src/controllers/payroll.controller.js</code>	Locks <code>Payroll.dailyRate</code> permanently upon payroll release
Batch DB Connection Throttling	<code>backend/src/controllers/payroll.controller.js</code>	Concurrent chunking (<code>chunkSize = 6</code>) optimizing pool resources
Two-Factor Physical Door Lock	<code>firmware/MaChip_Test/MaChip_Test.ino</code>	RFID tap + Biometric scan within 15s window (<code>TIMEOUT_2FA = 15000</code>)
Solenoid Relay Auto-Relock	<code>firmware/MaChip_Test/MaChip_Test.ino</code>	<code>SOLENOID_DURATION = 3000ms</code> auto-relock preventing tailgating
Hardware OTA Remote Flashing	<code>firmware/MaChip_Test/MaChip_Test.ino</code>	<code>ArduinoOTA</code> and <code>ESPmDNS</code> (<code>machip-esp32.local</code>) over LAN
Holiday API Fallback Service	<code>backend/src/utils/holidaySyncService.js</code>	Nager.Date REST API integration with static Philippine holiday list